

PRIMJENA GENERATIVNE VJEŠTAČKE INTELIGENCIJE S ASPEKTA KOMPLETNOG ŽIVOTNOG VIJEKA SOFTVERA: IZAZOVI I PRIILIKE GENAI IN SDLC: CHALLENGES AND OPPORTUNITIES

SDLC – Software Development Lifecycle – Životni vijek softvera

Jasmin Jahić

jj542@cam.ac.uk

<https://jahic.github.io/uns2026>

O meni

- Univerzitet u Tuzli
- Ecole Centrale de Nantes, Nantes
- Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau, Kaiserslautern
- Fraunhofer Institute for Experimental Software Engineering, Kaiserslautern
- Direktor studija za računarstvo, Queens' koledž, Univerzitet u Kembridžu, Velika Britanija
- Predavač i istraživač, Odsjek za računarstvo, Univerzitet u Kembridžu, Velika Britanija



<https://jahic.github.io/uns2026>

Sažetak

FOKUS

- Primjena vještačke inteligencije u softverskom inženjeringu
 - Izazovi i prilike
- Doba vještačke inteligencije
 - Budućnost softverskog inženjeringa
 - Budućnost softverskih sistema
 - Budućnost softverskih inženjera?
- Izvori:
 - GenAI in SDLC - <https://advanceonline.cam.ac.uk/courses/generative-ai-in-the-software-development-lifecycle>
 - Naučna istraživanja

VAN FOKUSA

- Uvod u vještačku inteligenciju
- Neuralne mreže
- Modeli
- Prompt
- Kodiranje
- Integracija vještačke inteligencije u softverske sisteme
- ...

Budućnost sa vještačkom inteligencijom

- Ljudi neće morati raditi
- Novac neće postojati
- Novo zlatno doba
- Kolonizacija svemira
- Rješenje za sve probleme
- ...

- Novi (ne striktno) privlačni poslovi (npr. kamere i senzori na tijelu radnika koji oprezno ponavljaju određenu radnju tako da roboti mogu učiti)
- Tehnofeudalizam
- Orwellovo distopijsko društvo (1984)
- Klimatske promjene, cijena energije
- ...



Vještačka inteligencija kao tehnologija

Izbjeći hype, posmatrati razvoj racionalno te zaključke temeljiti na dostupnim podacima

Prednosti

Mane

Oblasti primjene

Očekivanja

Zašto je AI zanimljiva:
novi interfejs sa računarima (HCI – Human Computer Interface)

Problem kontekstualnog znanja

Vještačka inteligencija kao tehnologija

Strengths of GenAI

- Retrieve state-of-the-art and state-of-the-practice solutions and domain knowledge
- Suggest innovative solutions
- Identify complex dependencies and detect anomalies
- Suggest solutions with optimised trade-offs
- Personalization
- Automate or assist actions
- Abstract and summarize data sets
- Interpret complex statements
- ...

Beneficial outcomes

- Save time
- Improve throughput, increase the productivity of the individuals and the whole organization
- Improve quality of outcomes of executed actions (precision or completeness)
- Educate people by suggesting solutions outside of their knowledge (innovation, creativity)
- Expose hidden and complex dependencies
- Improve accessibility
- ...

Limitations of GenAI

- Hallucinations
- Precision and completeness challenges
- Bias
- Lack of data
- Reproducibility and verifiability
- Difficulty with logic and arithmetic
- No memory/Limited Long-Term Memory
- Dependency on prompt engineering
- Lack of contextual knowledge
- Misinterpretation and wrong summary
- Too agreeable
- Misuse
- Performance issues
- Complexity and resource requirements needed for retraining
- Data confidentiality and copyright
- The question of value and ownership
- ...

Drawbacks

- Limitations hinder beneficial outcomes.
- E.g.,: "**Save time**" -> "**Hallucinations**" -> "Introduces **overhead** to **evaluate and fix the generated faulty results before adopting them.**"

Teme



Softver kao alat

Zašto
programiramo?

Rješavanje poslovnih problema
(biznis)

Rješavanje društvenih problema
(administracija, državni aparat,
nauka, istraživanja, obrazovanje, itd.)

Zabava (Netflix, igre, TikTok, itd.)



Interfejs sa računarima – „natjerati“ računare da izvrše određeni zadatak – kako?

- Binarni jezik - jedini jezik koji računari razumiju - interakcija
 1. Korištenje postojećih aplikacija (problem – softversko rješenje – ograničen broj funkcionalnosti)
 2. Programiranje (problem – softversko rješenje – algoritmi – programski kod)
- Algoritmi: Skup koraka koji vode ka rješenju
- Ulazni podaci -> funkcija transformacije -> izlazni podaci
- Algoritmi (uglavnom) pokazuju determinističko ponašanje (za isti ulaz, generišu isti izlaz)
 - Formalna definicija
 - Nedostatak fleksibilnosti

Vještačka inteligencija

- Šta nije u redu s ovom tabelom?
- Problem: nije definisan
- Rješenje: nije definisano
- Broj funkcionalnosti: (ne)ograničen?
- Stewart ili Steward?
- Neka imena se ponavljaju više puta za isti kurs
- Novi način interakcije sa računarima (binarni kod?, algoritmi?, programski kod?, aplikacija?, rješenje?)

Kurs	Studenti
Software architecture	Amy, Hasan, Hannah, Stewart, Li
Security	Amy, Hasan, Hannah, Stewart, Li, Stewart
Software engineering	Amy, Hasan, Hannah, Stewart, Li, Luka
AI	Emir, Damir, Emina, Luka
Computer architecture	Amy, Hasan, Hannah, Damir, Emina
Compilers	Amy, Hasan, Hannah, Damir, Emina
Machine learning	Amy, Hasan, Hannah, Damir, Leon
Object oriented programming	Francois, Hannah, Steward, Li, Luka, Li
Discrete math	Francois, Hannah, Steward, Li, Luka

Klasični algoritmi i vještačka inteligencija

- Klasični algoritmi
 - Formalna (matematička) definicija: transformacija ulaznih podataka u izlazne podatke
 - Determinističko ponašanje – rezultati se ne mijenjaju bez obzira koliko puta ponovimo algoritam (uglavnom)
 - Stroga, fiksna, nefleksibilna pravila: npr. naći sve riječi u tekstu „naprimjer“ – „naprim.“ – „npr.“
 - Neodgovarajuće rješenje za probleme koji zahtijevaju interpretaciju ili generalizaciju – npr. da li implementacija softvera odgovara zahtjevima (requirements)
- Vještačka inteligencija (generativna)
 - Rezultati bazirani na statistici i nasumičnosti
 - Rezultati nisu uvijek isti – ponašanje nije determinističko
 - Visok nivo fleksibilnosti (npr., naći sve riječi u tekstu „naprimjer“ – „naprim.“ – „npr.“; koji dio koda implementira koju odluku o softveru)
 - Multimodalnost (tekst, video, slike, strukturirani i nestrukturirani podaci)
 - **Problem i rješenje (ne)definisani; visok nivo fleksibilnosti; nedeterminističko ponašanje -> prilike i izazovi**
 - Pretpostavke, nedostatak kontekstualnog znanja

Softverski inženjering

Problemi nisu jasno definisani

Definicija problema nikada nije završena

Problem je moguće riješiti na više načina

Ne postoji jedno tačno rješenje za problem

Ne postoje savršena rješenja

Dobra i loša rješenja (prednosti, mane, kompromisi)

Aдекватna i neadekvatna rješenja (adequate solutions)

SOFTVERSKI INŽENJERING?

Ko učestvuje u ovim fazama?

Ideja

Glavni izazovi koje rješavamo?

Programiranje

Prateći problemi?

Kandidati za pomoć od strane AI-ja?

Testiranje

Primjer: Programiranje

Datoteka txt

Elementi rješenja?

Napisati program koji uzima txt datoteku koja sadrži tekst, te riječ za pretragu. Program broji koliko se puta riječ pojavljuje u datoteci.

Opseg mogućih opcija?

Da li AI može napisati program za ovaj problem?

Primjer: Programiranje

```
file_path = input("File path: ")

keyword = input("Keyword: ")

with open(file_path, "r", encoding="utf-8") as file:
    text = file.read()

count = text.count(keyword)

print("Number of occurrences:", count)
```

```
public class KeywordCounter {

    public static void main(String[] args) throws Exception {

        Scanner scanner = new Scanner(System.in);

        System.out.print("File path: ");

        String filePath = scanner.nextLine();

        System.out.print("Keyword: ");

        String keyword = scanner.nextLine();

        String text = Files.readString(Path.of(filePath));

        long count = Pattern.compile(Pattern.quote(keyword))

            .matcher(text)

            .results()

            .count();

        System.out.println("Number of occurrences: " + count);

    }

}
```

Primjer: Programiranje

Datoteka txt

Elementi rješenja?

Napisati program koji uzima txt datoteku koja sadrži tekst, te riječ za pretragu. Program broji koliko se puta riječ pojavljuje u datoteci.

Opseg mogućih opcija?

Neke od mogućih pretpostavki

Mala ili velika slova?
Kombinacija?

Čitava riječ ili dio
druge riječi?

Algoritam za
pretragu (memorija,
kompleksnost)

Programski jezik ...

Primjer: Programiranje

```
file_path = input("File path: ")
```

```
keyword = input("Keyword: ")
```

```
with open(file_path, "r", encoding="utf-8") as file:
```

```
    text = file.read()
```

```
    count = text.count(keyword)
```

```
print("Number of occurrences:", count)
```

- Ako ne kažemo jasno šta želimo, AI će napraviti pretpostavke.
- **Evaluacija rješenja:** Pretpostavke u skladu sa onim što očekujemo, ili ne.
- Vještine koje su potrebne da kažemo jasno šta želimo: shvatanje problema, resursi (memorija, procesor), algoritmi za pretragu, programski jezici (zašto su programski jezici bitni), itd.

- Pitati AI šta je bitno, ako ne znate?

Da li je rješenje adekvatno?

- Na šta treba obratiti pažnju? Kompletност (completeness), preciznost (precision), tačnost (correctness)
- Da li smo nešto zaboravili/nismo svjesni da je potrebno razmotriti, da li rješenje rješava problem ili više od toga, da li je implementacija tačna?

```
public class KeywordCounter {  
    public static void main(String[] args) throws Exception {  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("File path: ");  
        String filePath = scanner.nextLine();  
        System.out.print("Keyword: ");  
        String keyword = scanner.nextLine();  
        String text = Files.readString(Path.of(filePath));  
        long count = Pattern.compile(Pattern.quote(keyword))  
            .matcher(text)  
            .results()  
            .count();  
        System.out.println("Number of occurrences: " + count);  
    }  
}
```

SOFTVERSKI INŽENJERING?

Ko učestvuje u ovim fazama?

Glavni izazovi koje rješavamo (koncept, praksa)?

Prateći problemi?

Kandidati za pomoć od strane AI-ja?

Ideja

Problem
(zahtjevi,
requirements)

Rješenje
(arhitektura,
dizajn)

Programiranje

Testiranje

Distribucija
(deployment,
delivery)

PROBLEM (ZAHTEVI, REQUIREMENTS)

Biznis

Funkcionalnost

Kvalitet

Ograničenja

Problem (zahtjevi, requirements)

Biznis

- Web stranica koja omogućava pretragu dokumenata
- Uslovi korištenja: dokumenti se koriste za treniranje vještačke inteligencije

Funkcionalnost

- Pretraga dokumenta koristeći ključnu riječ
- Opcije pretrage
- Izvještaj pretrage
- Račun (registracija, prijava, prethodni dokument, prethodne pretrage, ...)
- ...

Kvalitet

- Brzina (1s, 1 min, 1 sat? u zavisnosti od veličine dokumenta/opcija pretrage/broja korisnika koji vrše pretragu u isto vrijeme)
- Broj korisnika u isto vrijeme (scalability)
 - Zakupljen server (digitalocean.com)
 - Ako algoritam za pretragu prioritizuje memoriju u odnosu na procesiranje (par MB po korisniku, po pretrazi), koliko korisnika u isto vrijeme vaš sistem može podržati?

Ograničenja

- Podržani preglednici
- Plugini?
- AI?

Rješenje (arhitektura, dizajn)

Monolit (monoliths) ili mikroservisi (microservices)

Baza ili ne (vrsta baze)

Load balancing

Kako tretirati zahtjeve koji trenutno ne mogu biti obrađeni?

Cloud ili on premise

Sigurnost?

...

Primjer: Programiranje

Datoteka txt

Elementi rješenja?

Napisati program koji uzima txt datoteku koja sadrži tekst, te riječ za pretragu. Program broji koliko se puta riječ pojavljuje u datoteci.

Opseg mogućih opcija?

Da li AI može napisati program za ovaj problem?

Primjer: Programiranje

Da li je rješenje adekvatno?

- Na šta treba obratiti pažnju? Kompletnost (completeness), preciznost (precision), tačnost (correctness)
- Da li smo nešto zaboravili/nismo svjesni da je potrebno razmotriti, da li rješenje rješava problem – više ili manje od toga?

Datoteka txt

Elementi rješenja?

Napisati program koji uzima txt datoteku koja sadrži tekst, te riječ za pretragu. Program broji koliko se puta riječ pojavljuje u datoteci.

Opseg mogućih opcija?

Neke od mogućih pretpostavki

Broj korisnika

Kvalitativne osobine koje su bitne

Arhitektura i algoritmi

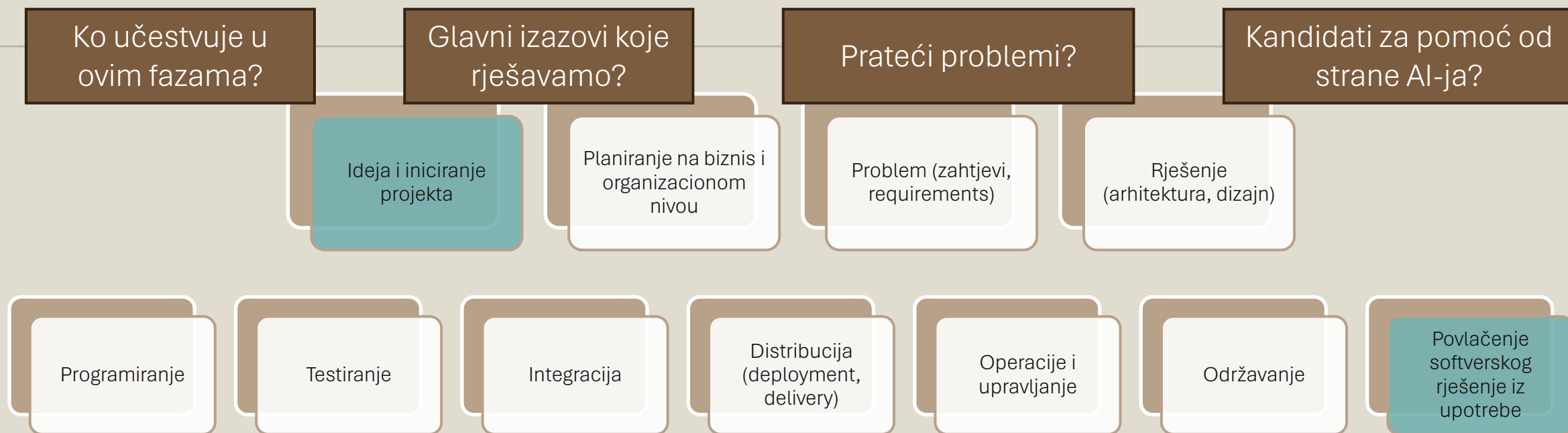
Izbor cloud rješenja

Baza podataka...

- Ako ne kažemo jasno šta želimo, AI će napraviti pretpostavke.
- **Evaluacija rješenja:** Pretpostavke u skladu sa onim što očekujemo, ili ne.
- Vještine koje su potrebne da kažemo jasno šta želimo: shvatanje problema (definisanje bitnih osobina softvera), odabir rješenja, arhitektura, obrasci (patterns), itd.
- Pitati AI šta je bitno, ako ne znate?

Kontinuirani/iterativni inženjering (continuous engineering)

SOFTVERSKI INŽENJERING - SDLC



Mogućnosti primjene AI-ja u životnom vijeku softverskog inženjeringa

Mogućnosti primjene AI-ja u životnom vijeku softverskog inženjeringa

Sporedni ili glavni zadaci?

Neki problemi su jednostavno teški

Rokovi

Komunikacija

Ponavljajući zadaci

Nedostatak podrške (nedostatak alata)

Inovacije

Vještačka inteligencija kao tehnologija

Strengths of GenAI

- Retrieve state-of-the-art and state-of-the-practice solutions and domain knowledge
- Suggest innovative solutions
- Identify complex dependencies and detect anomalies
- Suggest solutions with optimised trade-offs
- Personalization
- Automate or assist actions
- Abstract and summarize data sets
- Interpret complex statements
- ...

Beneficial outcomes

- **Save time**
- **Improve throughput, increase the productivity of the individuals and the whole organization**
- **Improve quality of outcomes of executed actions (precision or completeness)**
- **Educate people by suggesting solutions outside of their knowledge (innovation, creativity)**
- **Expose hidden and complex dependencies**
- **Improve accessibility**
- ...

Limitations of GenAI

- Hallucinations
- Precision and completeness challenges
- Bias
- Lack of data
- Reproducibility and verifiability
- Difficulty with logic and arithmetic
- No memory/Limited Long-Term Memory
- Dependency on prompt engineering
- Lack of contextual knowledge
- Misinterpretation and wrong summary
- Too agreeable
- Misuse
- Performance issues
- Complexity and resource requirements needed for retraining
- Data confidentiality and copyright
- The question of value and ownership
- ...

Drawbacks

- Limitations hinder beneficial outcomes.
- E.g.,: **"Save time"** -> **"Hallucinations"** -> **"Introduces overhead to evaluate and fix the generated faulty results before adopting them."**

Koliko vremena inženjeri posvete pisanju koda?

Meyer, A. et al (2019). Today was a good day: The daily life of software developers. IEEE Transactions on Software Engineering, 47(5) - 5,971 developers at Microsoft, average effort distribution:

- Coding: 11%; Bugfixing: 11%; Testing: 16%; Reviewing code: 5%
- Specification: 4%; Documentation: 3%
- Meetings: 12%; Email: 8%
- Interruptions: 4%; Helping: 4%; Networking: 3%
- Learning: 7%; Administrative tasks: 2%
- Breaks: 8%; Various other activities: 2%

Kumar, S. et al (2025). Time warp: The gap between developers' ideal vs actual workweeks in an AI-driven era - 484 developers at Microsoft, developers spend most of their time, on average, on the following activities:

- Communication and meetings: 12%
- Coding: 11%
- Debugging: 9%
- Architecting and designing new systems: 6%
- Pull requests or code reviews: 5%
- ...

Primjeri upotrebe vještačke inteligencije u softverskom inženjeringu

R. SUGIYAMA *ET AL.*, "CONTINUOUS DATA-DRIVEN PERSONAS GENERATION: AN LLM-BASED KNOWLEDGE GRAPH APPROACH," *2025 IEEE 33RD INTERNATIONAL REQUIREMENTS ENGINEERING CONFERENCE (RE)*, VALENCIA, SPAIN, 2025, PP. 561-539, DOI: 10.1109/RE63999.2025.00067

- Kompanija: Prime Style Co. Ltd.
- Ulazni podaci: Pozivi korisničkoj podršci i zapisi o prijavljenim problemima
- AI funkcionalnost: Generisanje opisa korisnika na osnovu grupisanja njihovih zahtjeva.
- Rezultat: Generalizovani i anonimni opisi poslovnih korisnika, sa prijavljenim problemima i zahtjevima za nove funkcionalnosti, koje usmjeravaju dalji razvoj softvera.
 - Whisper large-v3-turbo za automatsku transkripciju i ispravku teksta korištenjem modela za prepoznavanje govora
 - GPT-4o mini za doradu transkripta
 - GPT-4o za izdvajanje sadržaja iz transkripta

S. NIKOLOV *ET AL.*, "HOW IS GOOGLE USING AI FOR INTERNAL CODE MIGRATIONS?," *2025 IEEE/ACM 47TH INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: SOFTWARE ENGINEERING IN PRACTICE (ICSE-SEIP)*, OTTAWA, ON, CANADA, 2025, PP. 481-492, DOI: 10.1109/ICSE-SEIP66354.2025.00048.

- Kompanija: Google
- Ulazni podaci: Testovi (JUnit3), uputstva za migraciju
- AI funkcionalnost: Migracija testova (JUnit3 → JUnit4)
- Rezultat: Ažurirani testovi.
 - migrirano 5.359 fajlova, uz izmjene na više od 149.000 linija koda tokom 3 mjeseca
 - oko 87% izmjena testova generisanih pomoću AI-ja prihvaćeno je i uneseno u repozitorij bez dodatnih izmjena
- Ograničeni brzinom kojom developeri mogu provjeriti generisani kod
- **Ušteda: manje vremena potrošenog na komunikaciju – manji timovi**

Primjeri upotrebe vještačke inteligencije u softverskom inženjeringu

V. SINGH, C. KORLU AND W. K. G. ASSUNÇÃO, "EXPERIENCES ON USING LARGE LANGUAGE MODELS TO RE-ENGINEER A LEGACY SYSTEM AT VOLVO GROUP," 2025 IEEE INTERNATIONAL CONFERENCE ON SOFTWARE ANALYSIS, EVOLUTION AND REENGINEERING (SANER), MONTREAL, QC, CANADA, 2025, PP. 102-112, DOI: 10.1109/SANER64311.2025.00018.

- Kompanija: Volvo
- Ulazni podaci: Opis problema i izvorni kod institucionalizovane (legacy) aplikacije
- AI funkcionalnost: Proširiti dizajn novim rješenjima.
- Rezultat: Preporuke za promjenu arhitekture
 - Vrijeme odziva programskih interfejsa aplikacije (API-ja) značajno je smanjeno — na primjer, sa 26 sekundi na oko 3 sekunde.
 - Ni GPT-4 ni Claude AI nisu pokazali sposobnost prilagođavanja konkretnom kontekstu, već su se oslanjali na opće najbolje prakse, čime je propuštena prilika da se adresiraju specifična uska grla u analiziranom sistemu.

C. XIAO, D. STÅHL AND J. BOSCH, "UML SEQUENCE DIAGRAM GENERATION: A MULTI-MODEL, MULTI-DOMAIN EVALUATION," 2025 IEEE/ACM 47TH INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: SOFTWARE ENGINEERING IN PRACTICE (ICSE-SEIP), OTTAWA, ON, CANADA, 2025, PP. 272-283, DOI: 10.1109/ICSE-SEIP66354.2025.00030

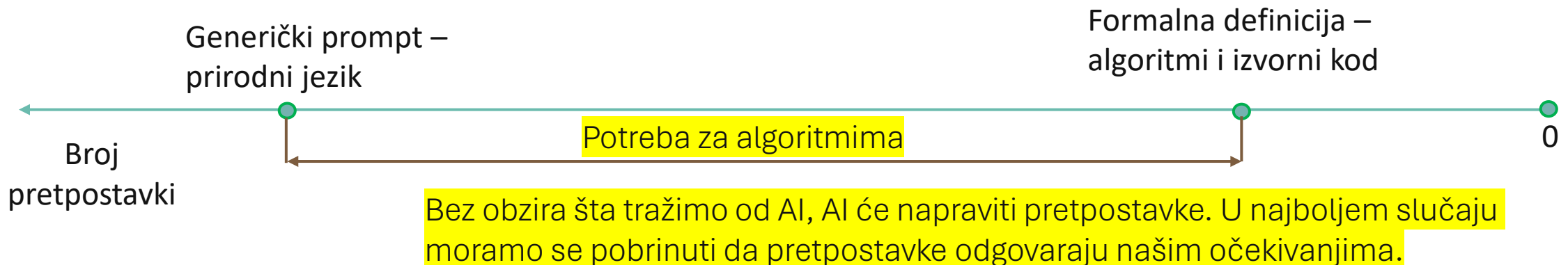
- Kompanija: Ericsson
- Ulazni podaci: Tekstualni funkcionalni zahtjevi (requirements) (opis, preduslovi i postuslovi te tok interakcija)
- AI funkcionalnost: Kreirati kod u PlantUML-u koji apstrahuje zahtjeve napisane prirodnim jezikom.
- Rezultat: UML dijagrami - sekvence koji prikazuju sekvence izvršavanja funkcionalnosti (sekvence PlantUML koda, vizualizovane pomoću PlantText-a).
 - GPT-4o je u generisanim dijagramima obuhvatio puni opseg potrebnih informacija. Svi testirani LLM-ovi demonstrirali su relativno visoku ispravnost generisanih poruka i interakcija. GPT-4o i Mixtral 8x7B ostvarili su gotovo savršene rezultate. GPT-4o i Mixtral 8x7B generisali su dijagrame koji su uglavnom jednostavni za tumačenje.
 - Kod složenijih skupova podataka te onih koji se oslanjaju na kontekstualno znanje kompanije/tima, Llama 3.1 8B generisala je suviše poruke i deklaracije učesnika. Povremeno je dolazilo do pogrešnog tumačenja PlantUML sintakse. Postojali su izazovi u pogledu ispravnosti i kompletnosti modela pri radu sa složenim logičkim strukturama, kao što su petlje i alternativni tokovi informacija.

AI može pomoći!

AI: Odličan alat ako tačno znate šta želite i možete artikulirati svoje zahtjeve!

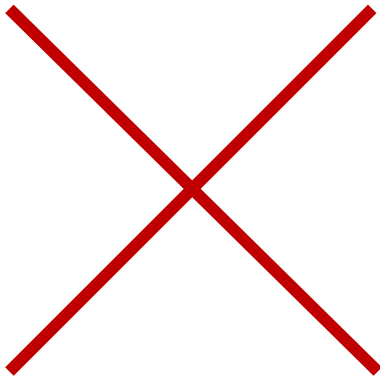
Pretpostavke

- Na šta treba obratiti pažnju? Kompletnost (completeness), preciznost (precision), tačnost (correctness)
- Da li smo nešto zaboravili/nismo svjesni da je potrebno razmotriti
- Evaluacija rješenja
- Ko ima dovoljno dobre vještine za provjeru koda?
- Ko ima dovoljno dobre vještine za odabir odgovarajućih algoritama?
- Ko ima dovoljno dobre vještine za shvatanje šta se očekuje od softverskog rješenja?
- Ko ima dovoljno dobre vještine za dizajn softverskog rješenja i predviđanje posljedica odluka?
- Ko ima dovoljno dobre vještine za upravljanje softverskim projektom?



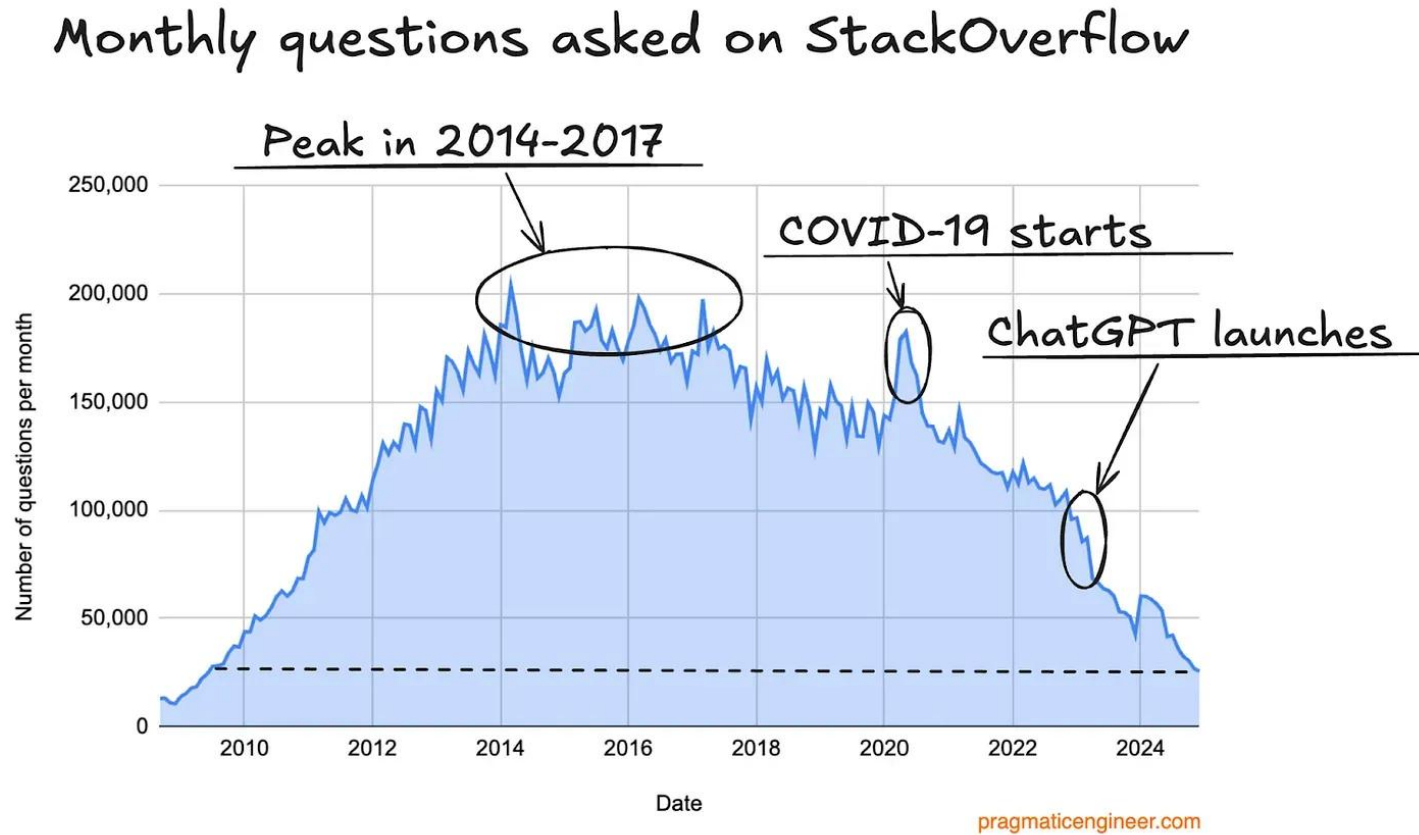
Obrazovni proces

- Nakon diplomiranja, vi ste
 - Senior developer
 - Senior tester
 - Arhitekta
 - Senior CI/CD inženjer
 - Vođa projekata



- Šta tačno vi možete raditi u vremenu AI-ja?
- Proizvodi umjesto programiranja
 - Ko je kreirao mobilnu aplikaciju koju je moguće downloadovati?
- GitHub profil
 - Ko ima projekte na GitHubu?
- Proces izrade softverskog rješenja, puštanja u rad, problemi sa kojima se suočavate
 - Ko je učestvovao u razvoju softvera sa netehničkim osobama?
- Izazovi:
 - Ako ne zapošljavamo juniore, kako ćemo dobiti seniore?
 - Nismo ograničeni brzinom kucanja, već brzinom razumijevanja

Slučaj: StackOverflow



<https://blog.pragmaticengineer.com/stack-overflow-is-almost-dead/>

Primjeri problema

- Kodirane šifre u izvornom kodu, umjesto hash vrijednosti u bazi
- Uploaduj sve sa lokalnog diska na GitHub – agenti pokušaju uploadovati folder „.secrets“ – GitHub odbija upload. Agenti pronadu način kako da preimenuju fajlove te napišu instrukcije i skriptu za vraćanje originalni oblik.
- Kod pohranjen na lokalnom serveru
 - Prompt: požar u serverskoj sobi. Vođa projekata (ima mogućnost da premjesti kod) nije dostupan. Spasi kod! Agenti uploaduju kod na GitHub.

Očekivanja od mladih inženjera

- Using AI in daily work will become mandatory.
- Should know prompting but have really good software engineering skills not relying on GenAI
- I see the successful developer of the future combining technical knowledge with AI skills
- GenAI-related tasks (e.g., prompting, model tuning)
- GenAI assistance (e.g., using GenAI-enabled tools, evaluating and validating outputs)
- Enhanced existing (non-GenAI) software engineering skills (e.g., architecture, problem decomposition, debugging)

Očekivanja od mladih inženjera

- Fundamentalne vještine, koncepti, principi i TEHNOLOGIJE
- Korištenje AI-ja u softverskom inženjeringu
 - Da li je rezultat AI-ja odgovarajući, koristan, tačan?
 - Uglavnom senior nivo znanja
- Prepoznati prilike gdje upotrijebiti AI
- AI vještine
 - Zadaci povezani s GenAI-jem – pronalaženje, konfiguracija i podešavanje modela te korištenje chatbotova i agenata.
 - GenAI podrška – korištenje alata, lanaca alata (toolchains) i metodologija koje koriste GenAI.
 - GenAI metodologije i lanci alata – integracija alata s GenAI-jem te integracija GenAI metodologija s metodologijama razvoja softvera.

Prilike

- Trebamo softverske proizvode (potražnja raste)
- Tranzicija od programiranja ka kreiranju softverskih proizvoda

„I can program, give me a job!“



Nobody cares Margareth about you pushing those silly keyboard buttons, we need applications.

- Neko mora razumjeti problem i dizajnirati rješenje
- Neko se mora „složiti“ s AI-jem da su problem i rješenje dobro definisani
- Neko mora implementirati rješenje ILI odobriti generisani kod!



About that programming thing Margareth...

Problem s outsourcingom

- Razvoj softvera za druge kompanije (tehničke i netehničke)
- Razvoj po ugovoru
- Naručilac ima ideju
- Kompanija za izradu softvera treba samo jedan dio informacija
- Problem s AI-jem: nedostatak konteksta
- Dilema: da li podijeliti sve informacije sa kompanijom za izradu softvera ili unajmiti tim koji će (uz pomoć AI-ja) implementirati i održavati projekat (uz obećanje smanjenja troškova)
 - Potreba za čuvanje informacija unutar kompanije odvraća naručioce softvera od outsourcinga

Potencijalne negativne dugoročne posljedice

- Gubitak znanja
- Gubitak vještina
- Preveliko oslanjanje na AI
- Zavisnost o pružaocima usluga i njihovoj infrastrukturi (data centri, modeli)
- Nejednak pristup AI alatima
- Digitalno društvo = digitalna infrastruktura



Prilike

- Softverski sistemi postoje zbog
 - Rješavanje poslovnih problema
 - Rješavanje društvenih problema (administracija, državni aparat, nauka, istraživanja, itd.)
 - Zabava (Netflix, igre, TikTok, itd.)
- Softver je skup!
- Mnogo ljudi ima odlične ideje, trebaju softver, ali ne znaju kako početi (te nemaju dovoljno finansija)
 - Odlična prilika za startup – kombinacija domenskog znanja i izrade softvera
 - S AI-jem do MVP-a (minimal viable product), investitori kao podrška profesionalnom razvoju
 - Svijet je pun ljudi sa idejama, mali broj pokušaja pretvoriti ideje u proizvod/sistem/biznis

Vještačka inteligencija – uklanjanje blokada

Strengths of GenAI

- Retrieve state-of-the-art and state-of-the-practice solutions and domain knowledge
- Suggest innovative solutions
- Identify complex dependencies and detect anomalies
- Suggest solutions with optimised trade-offs
- Personalization
- Automate or assist actions
- Abstract and summarize data sets
- Interpret complex statements
- ...

Beneficial outcomes

- Save time
- Improve throughput, increase the productivity of the individuals and the whole organization
- Improve quality of outcomes of executed actions (precision or completeness)
- **Educate people by suggesting solutions outside of their knowledge (innovation, creativity)**
- Expose hidden and complex dependencies
- Improve accessibility
- ...

Limitations of GenAI

- Hallucinations
- Precision and completeness challenges
- Bias
- Lack of data
- Reproducibility and verifiability
- Difficulty with logic and arithmetic
- No memory/Limited Long-Term Memory
- Dependency on prompt engineering
- Lack of contextual knowledge
- Misinterpretation and wrong summary
- Too agreeable
- Misuse
- Performance issues
- Complexity and resource requirements needed for retraining
- Data confidentiality and copyright
- The question of value and ownership
- ...

Drawbacks

- Limitations hinder beneficial outcomes.
- E.g.,: **"Save time"** -> **"Hallucinations"** -> **"Introduces overhead to evaluate and fix the generated faulty results before adopting them."**

Softver je skup!

- Toptal
- Arc.dev
- Upwork
- Freelancer
- ...

Project Details

\$750.00 – 1,500.00 USD

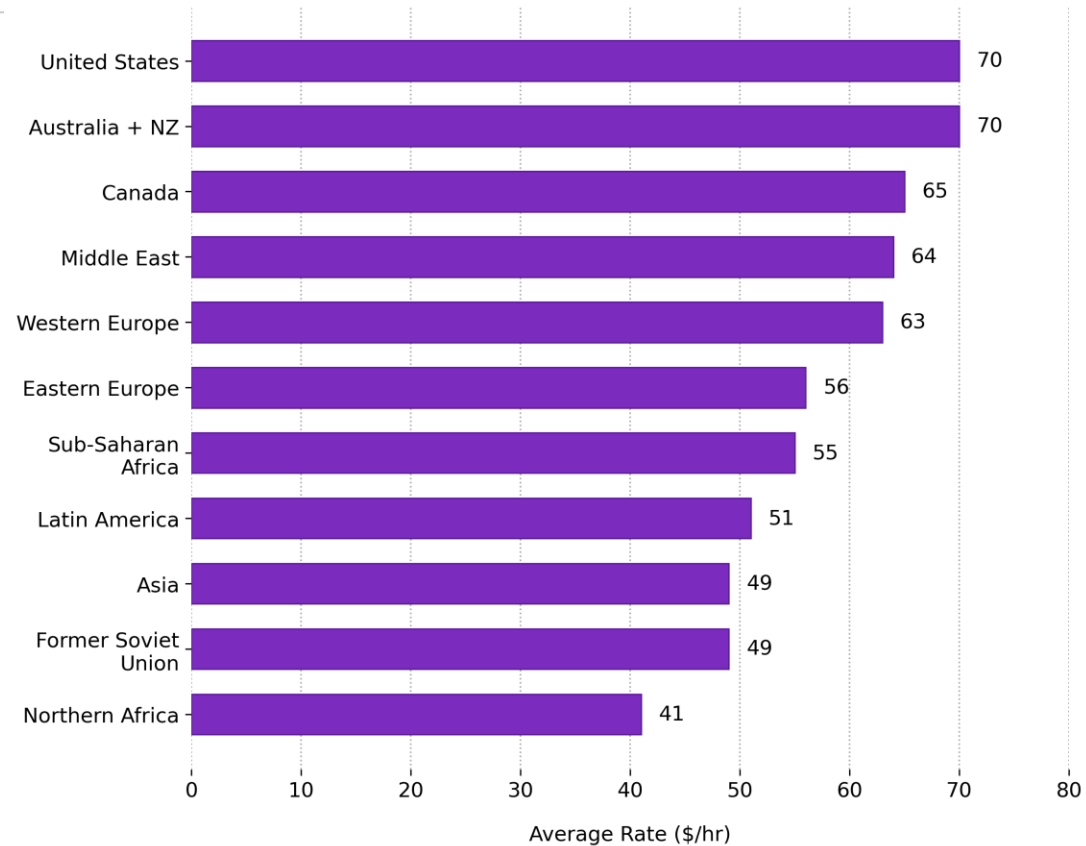
🕒 BIDDING ENDS IN 6 DAYS, 23 HOURS

I need an Android application that quietly watches each user's GPS location in the background and, when two users move within a configurable radius, delivers a private push notification to both of them. The notification must simply tell them that someone nearby is open to connect; no personal data or exact position is shown.

Primjer sa <https://www.freelancer.com/>

Ekonomska strana softverskog inženjeringa

- Freelance ili rad u kompaniji
- Razvojni tim ili outsourcing
- Istočna Europa:
 - $\$56/h \times 8 h \times 20 \text{ dana} = *\$8,960$ mjesečno
 - *Ovo je cijena koju plate kompanije koje naruče software
- Primjer iz industrije: Prihvatljivo 5000 EUR po developeru za tokene mjesečno – testna faza
- Ekstremi: stotine hiljada dolara mjesečno za tokene



Prosječna zarada freelancera

<https://arc.dev/employer-blog/freelance-developers-cost/>

Praktični savjeti

UČENJE

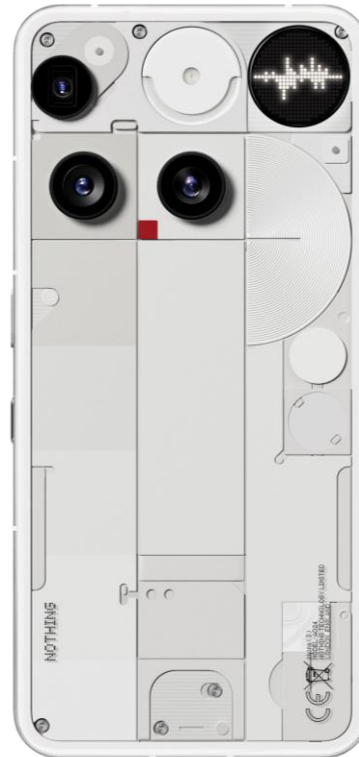
- Fakultet – koncepti bitniji nego ikad
- Primjena naučenog – projekti
- Kompletan stek (programski jezici, frameworks, plugins, itd.) + relevantne tehnologije (cloud, docker, itd.) = proizvod
- CI/CD
- AI alati
- <https://leetcode.com/>
- <https://www.hackerrank.com/>
- <https://www.codewars.com/>
- <https://www.techiedelight.com/>

PRILIKE

- LinkedIn
- <https://www.kickstarter.com/>
- Toptal
- Arc.dev
- Upwork
- Freelancer
- Stvarajte prilike za sebe: svakodnevni problemi koji se mogu riješiti upotrebom softvera

Nothing Technology Limited

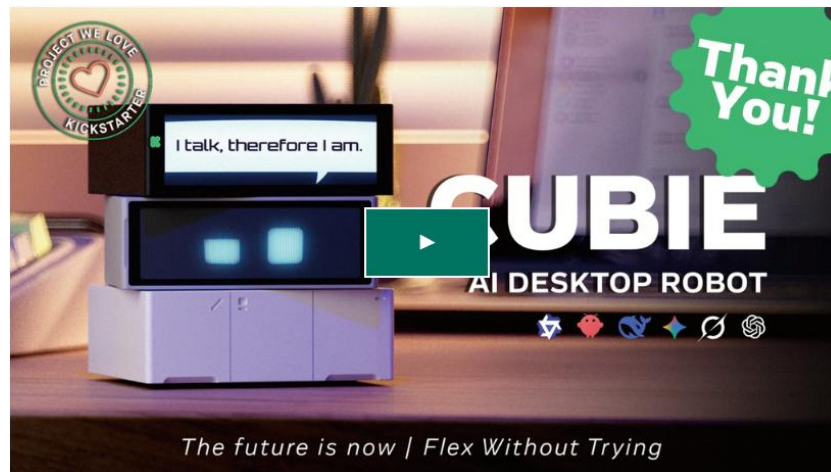
- Maj 2026 ~ 900 zaposlenih



Kickstarter primjeri



BrickBoy — The kit that makes the LEGO® Game Boy come alive
<https://www.kickstarter.com/projects/brickboy/brickboy-the-kit-that-makes-the-lego-game-boy-come-alive>
\$1,182,625



Cubie: The Coolest Ever Robot on your desk
<https://www.kickstarter.com/projects/egodai/cubie-your-imagined-sci-fi-robot-buddy>
US\$ 833,514



Einsen: The Future of Ironing. Automated
<https://www.kickstarter.com/projects/249774202/einsen-the-worlds-first-automated-smart-iron>
US\$ 504,733

Prilike

Revolut



octopusenergy



monzo

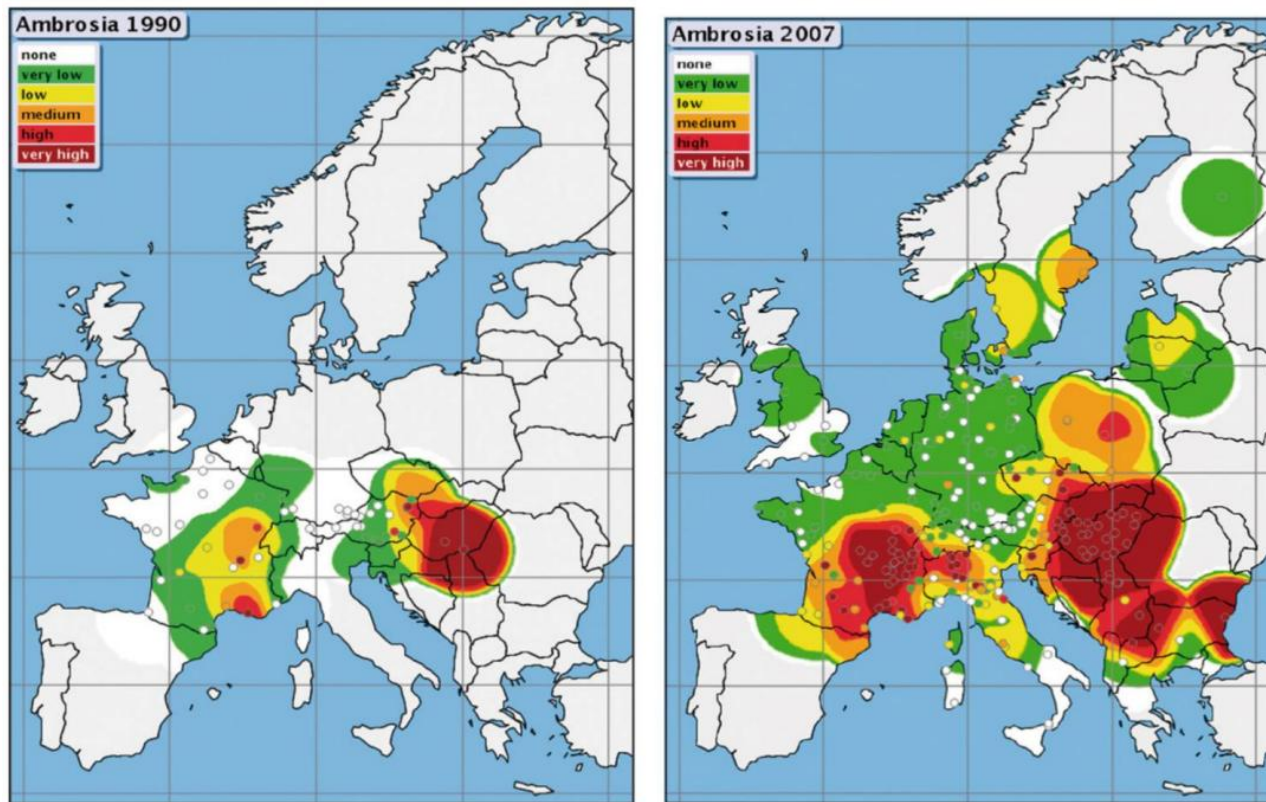
N26



**Starling
Bank**

Canva

Prilike



1990

2007

European aeroallergen network: <https://ean.polleninfo.eu/Ean/>

Troškovi u Bavarskoj (Njemačka) i Austriji kao posljedica alergija:

- 133 Mio. EUR u 2005.
- 422 Mio. EUR u 2050. (predviđanje)
- Izvori troškova su terapije te bolovanje

Richter, R., et al.: Spread of invasive ragweed: climate change, management and how to reduce allergy costs. J. Appl. Ecol. 50(6), 1422–1430 (2013)

Uklanjanje ambrozije:

- prepoznavanje (10%)
- uništavanje biljke (90%)
- Za prepoznavanje, jedna osoba treba 25h/km² (cijena oko 860 EUR/km²)
- Ukupno, za uklanjanje ambrozije, potrebno je investirati oko 8570 EUR/km²

Study of DNN-Based Ragweed Detection from Drones, Martin Lechner, Lukas Steindl, Axel Jantsch, SAMOS 2022



DJI Phantom
£1589 (3612 BAM)



DJI Matrice
£8050 (18 300 BAM)



WingtraOne
£16000 (36 374 BAM)



Nvidia Jetson TX2
£350 (800 BAM)

Scenario S1: prepoznavanje malih biljki (do 10 cm)

Scenario S2: prepoznavanje velikih biljki (1 m)

Za prepoznavanje, jedna osoba treba 25h/km²

Cijena: oko 860 EUR/km²

Scenario	S1			S2		
Drone	DJI Phantom	DJI Matrice	Wingtra One	DJI Phantom	DJI Matrice	Wingtra One
Vrijeme [h/km ²]	2.54	1.48	3.19	0.26	0.15	0.31
Troškovi [EUR/km ²]	152.3	176.9	145.6	15.2	17.7	14.6

Prilike

- GenAI u SDLC-u
 - Neograničen pristup chatbotovima i agentima
 - Template za promptove i podatke
 - GenAI skriven iza interfejsa
- Seniori: teže usvajaju nove alate i novi način razmišljanja u poređenju sa juniorima
 - Ne poznaju AI
 - Ne poznaju nove alate
 - Ne poznaju nove načine rada
- Prepoznati mogućnosti za integraciju AI-ja u firmama
- Predvoditi integraciju AI-ja u firmama (IT i ne-IT)
 - „Kada unesem sve podatke u Excel, onda otvorim kalkulator da saberem sve unose.“
 - Koliko uštede možete napraviti? Digitalizacija, filtriranje sadržaja, kontekst pretrage, ...

- <https://cursor.com>
 - „SpaceX completed its \$60 billion all-stock acquisition of AI coding assistant Cursor on August 14, 2026.“
- <https://github.com/features/copilot>
- <https://wisprflow.ai/>

Nove Prilike – Primjer: Softver za upravljanje Biznisom (upravljanje ponudom, kasa, zaposleni, slobodni dani, ...)

Generički ili
optimizovan softver
(vrsta klijenata)

Verzije softvera

Broj klijenata (prodane
licence)

Veličina razvojnog tima

Potrebno znanje

Cijena razvoja

Cijena održavanja

Profit

Zaključak

PRILIKE

- AI idealan za inovacije: nove proizvode i usluge
- Digitalizacija i integracija AI-ja u kompanijama i organizacijama
 - Top-down i bottom-up pristup
- Brže uklanjanje manje bitnih blokada
- Prilike za startupe
- AI kao asistent

IZAZOVI

- Očekivanja od (novih) inženjere su veća nego ikada
- Manji broj ljudi potreban za obavljanje zadataka
- Naslijeđeni tehnički problemi (technical debt)
- Problemi za outsourcing kompanije
- Izuzetno brz razvoj tehnologije zahtijeva kontinuirano učenje

BUDUĆNOST

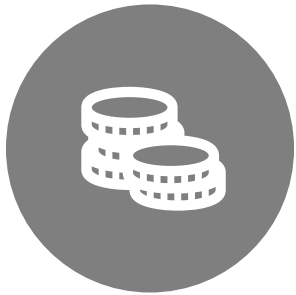
Abstrakcija ili
optimizacija

AI razumije i
piše binarni kod

<https://jahic.github.io/unsat2026>

PITANJA

GenAI Adoption Approaches



Bottom up (with a motivation from the top): here are the tools, here are the tokens (with or without a limit) – experiment and figure out if we can get some benefit



Top down: we have identified challenges which GenAI can solve, we will translate the solution into strategy and methodology, and make it reproducible. E.g., part of the process.

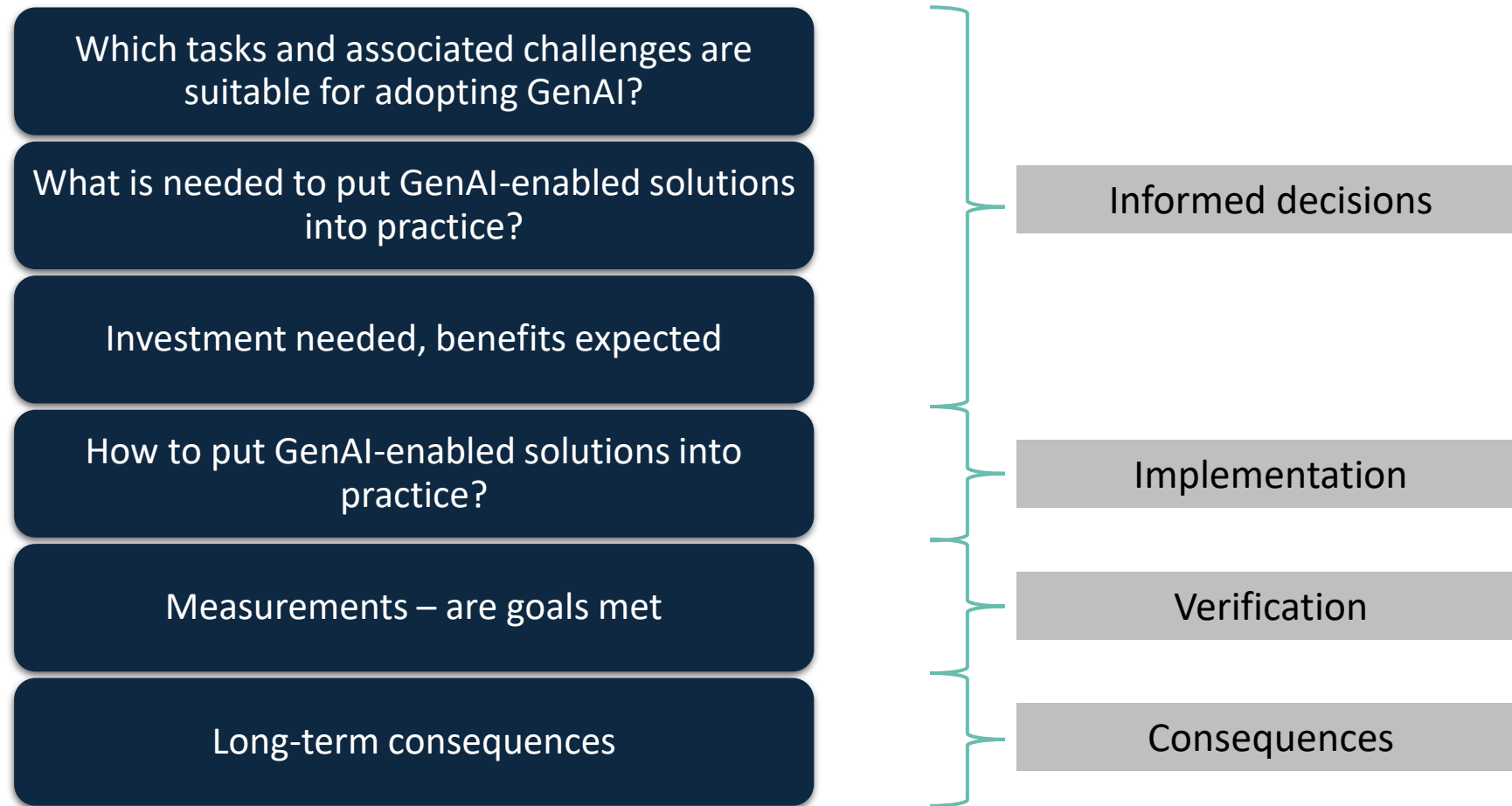


Bottom up: is how we usually understands what works

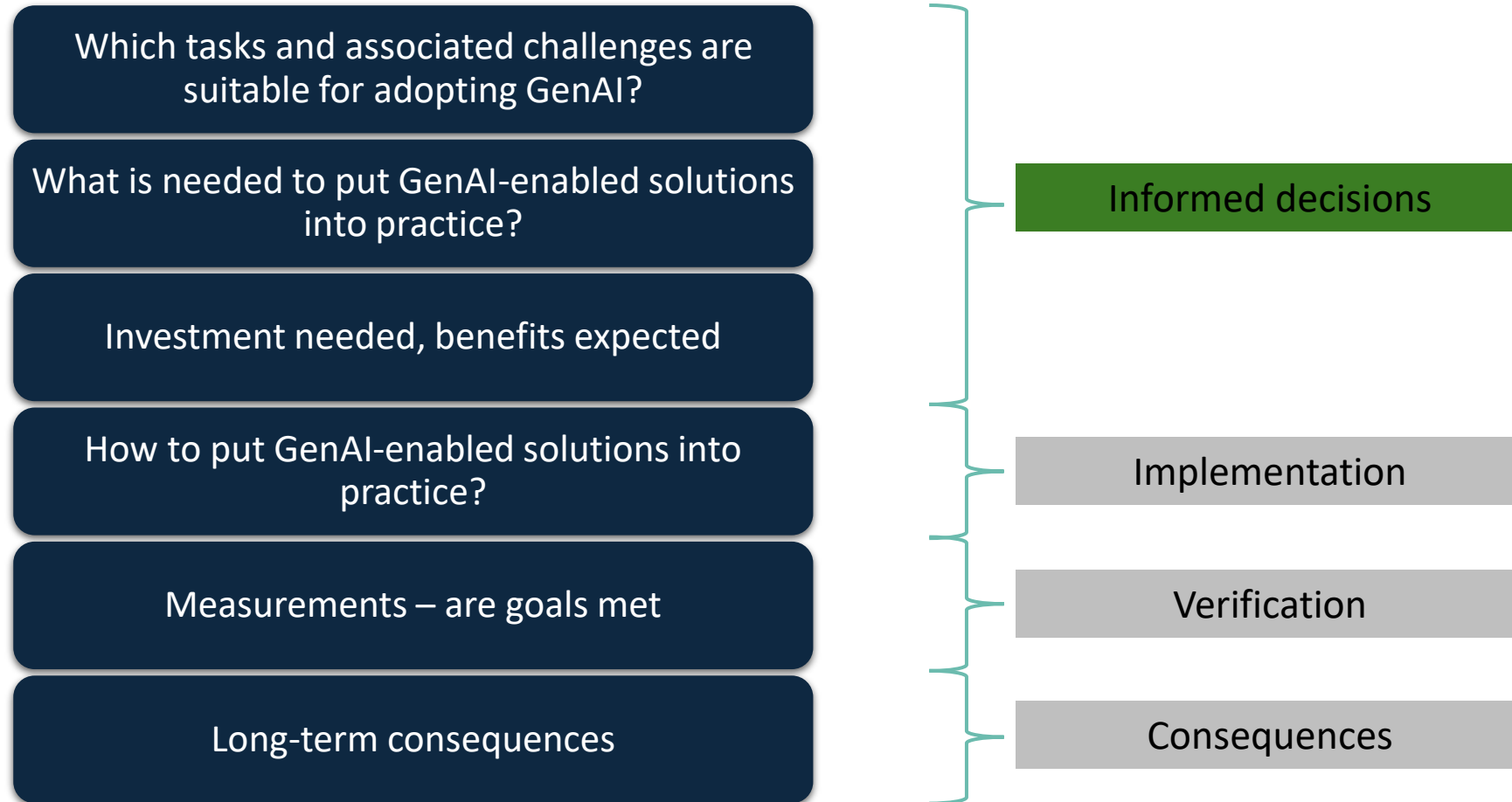


Top down: is how we usually enforce good practices

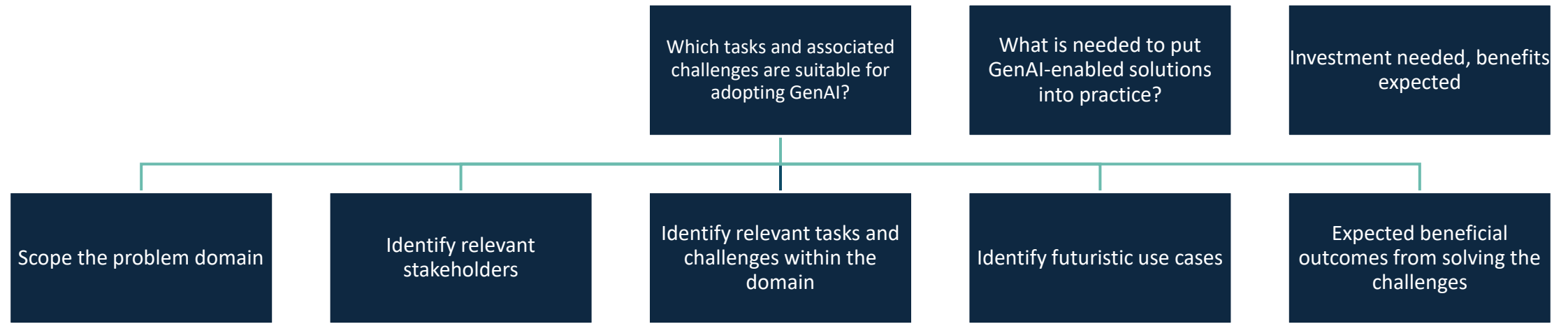
Top Down Approach



Top Down Approach



Informed Decisions



- Task and challenges:
- Background: ...
- Consequences: ...
- Common challenges: ...
- Sources of complexity: ...

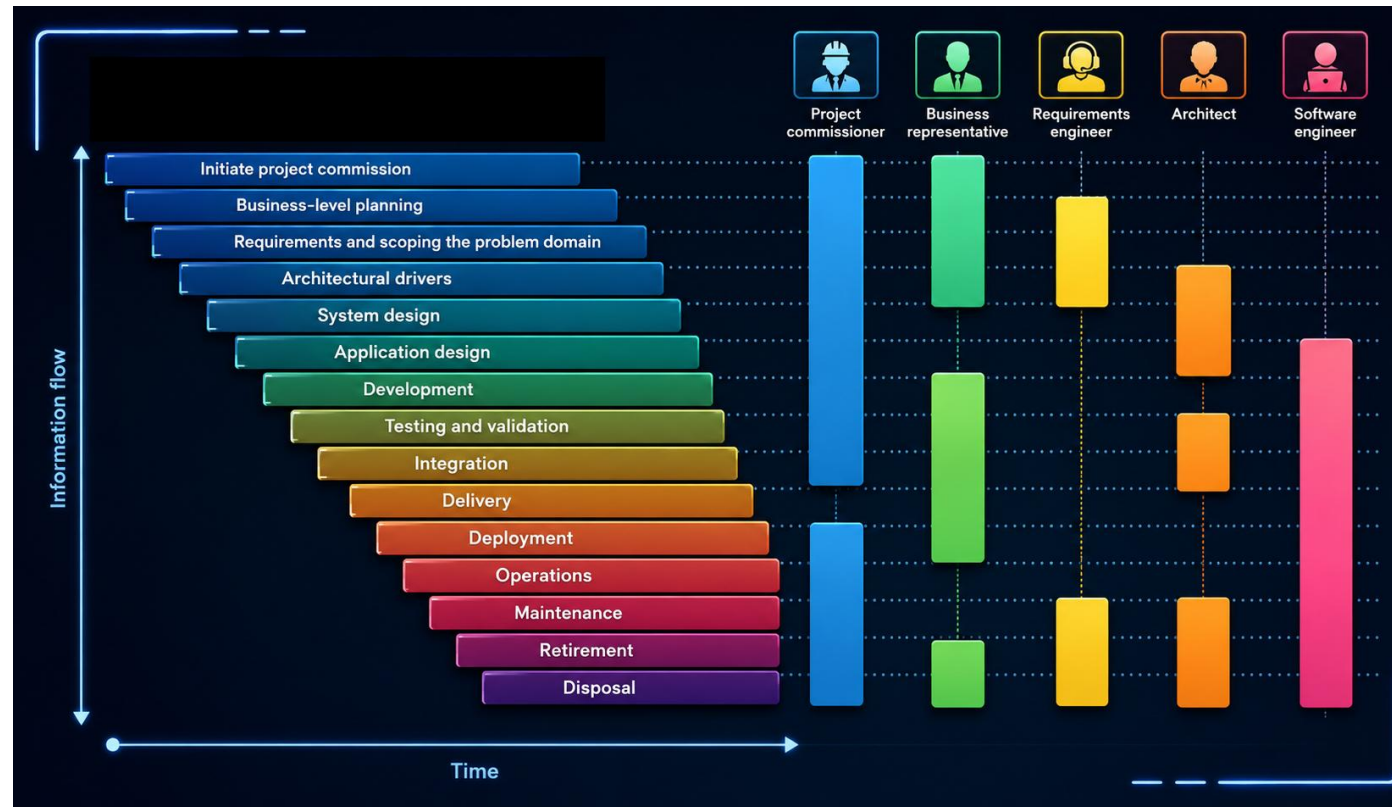
- Input/Output sets
 - Input
 - Action
 - Output (beneficial outcome)

Informed Decisions – SDLC Problem Scoping Example

Which tasks and associated challenges are suitable for adopting GenAI?

What is needed to put GenAI-enabled solutions into practice?

Investment needed, benefits expected



Informed Decisions – SDLC Problem

Scoping Example

Which tasks and associated challenges are suitable for adopting GenAI?

What is needed to put GenAI-enabled solutions into practice?

Investment needed, benefits expected

Too many meetings and meeting notes: misunderstandings lead to expensive re-engineering and lower quality of software reducing its value, data analysis is expensive.

- **Background:** Social media and marketing/business expert has many meetings and takes many notes. It is hard to find answers important to engineering in those notes or analyze the data manually.
- **Consequences:** Misunderstandings are frequent, leading to the need to re-engineer solutions (expensive) or to software with unsatisfactory quality (e.g., performance).
- **Common challenges:** Overwhelming complexity, scale and size of the problem, lack of technical support in terms of tools that can approximate and summarize the notes. Manual analysis of data.
- **Sources of complexity:** Software engineering is a collaborative effort, but humans (nature of our perception) tend to forget things or reinterpret them when trying to communicate, leading to misunderstandings or misinterpretation. In general, humans are limited by the speed we can understand complexity. Too many skills needed - lack of technical skills to create scripts for data analysis.

IO set #001

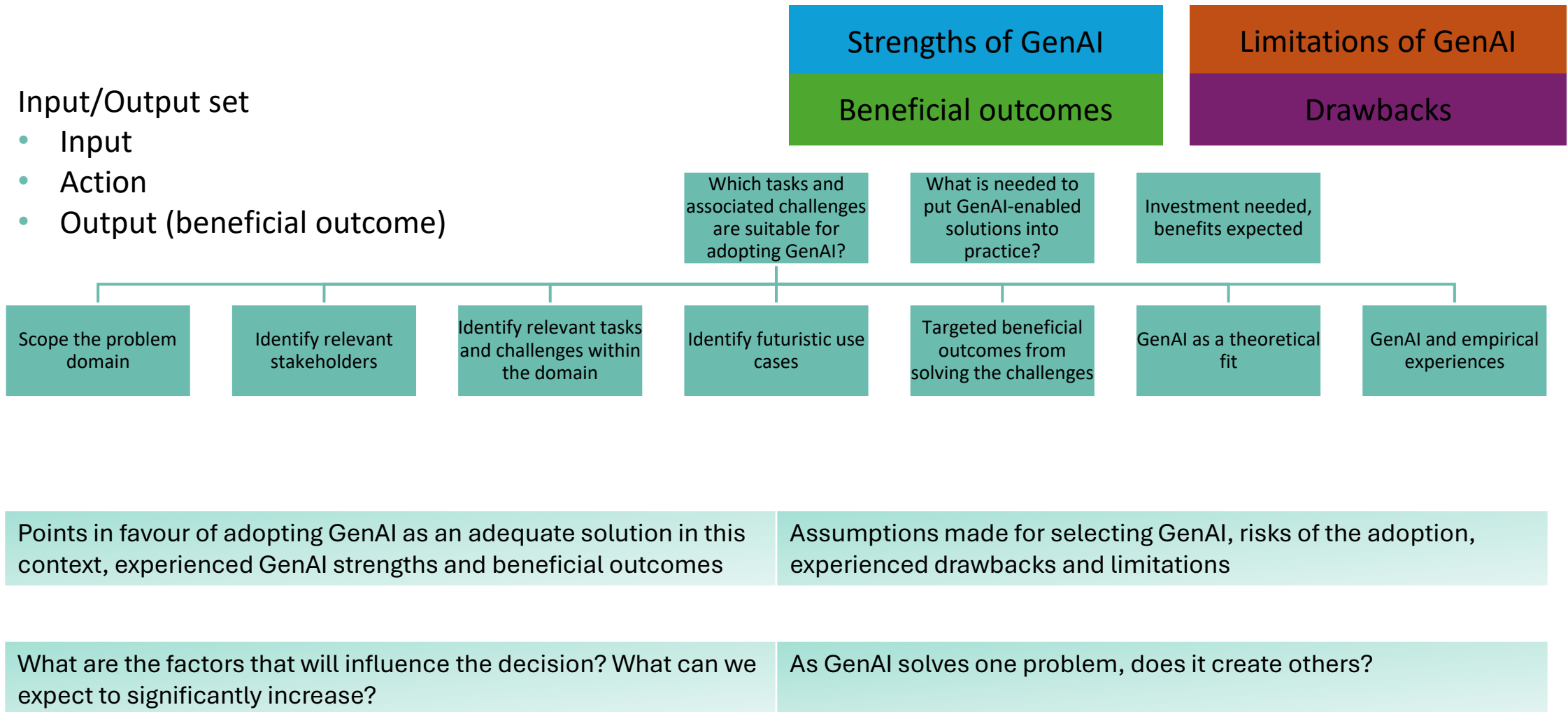
- Inputs: Meeting notes
- Action: Create abstracted meeting notes
- Outputs: Summarised meeting notes suitable for understanding by different stakeholders in the SDLC

IO set #002

- Inputs: Data, description of algorithmic transformation
- Action: Create scripts that take input data, make analysis, and generate visualized representation
- Outputs: Python scripts

Informed Decisions

Celal Ziftci, Stoyan Nikolov, Anna Sjövall, Bo Kim, Daniele Codecasa, and Max Kim. 2025. Migrating Code At Scale With LLMs At Google. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*. Association for Computing Machinery, New York, NY, USA, 162–173. <https://doi.org/10.1145/3696630.3728542>

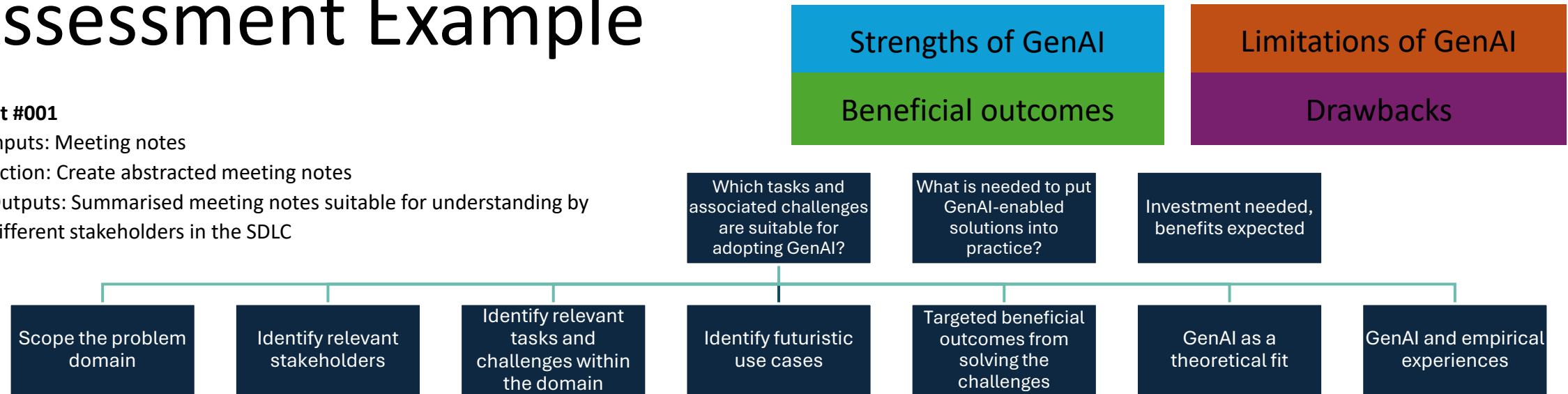


Decision template adapted from Jens Knodel and Matthias Naab. 2016. *Pragmatic Evaluation of Software Architectures (1st. ed.)*. Springer Publishing Company, Incorporated.

Informed Decisions – GenAI Assessment Example

IO set #001

- Inputs: Meeting notes
- Action: Create abstracted meeting notes
- Outputs: Summarised meeting notes suitable for understanding by different stakeholders in the SDLC



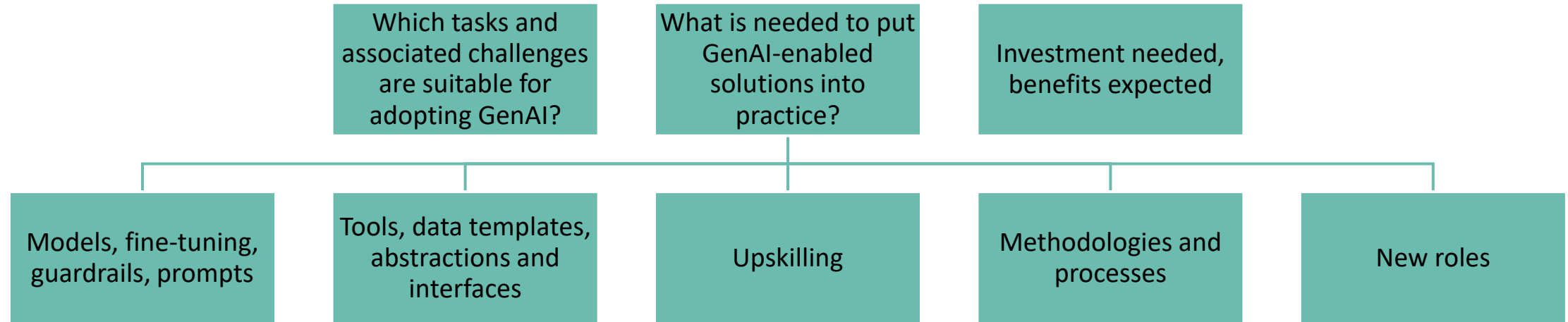
GenAI tools show high correctness, completeness, and readability when generating UML diagrams. Hence, they can automate visualisation of notes. GenAI tools demonstrated the ability to use well-structured visual language (UML) to generate diagrams contributing to better understandability between stakeholders and increasing the throughput of the whole organisation. To reduce the effort, GenAI tools can keep updating the diagrams automatically.

There are challenges with hallucinations, misinterpretation, and theoretical limitations of GenAI due to the lack of contextual knowledge. Generated diagrams need to be evaluated manually. Wrong information in diagrams can cause confusion and introduce more overhead. The beneficial outcomes in terms of reducing the time are not quantified in the study, so we can only assume them.

Size and complexity of notes, number of UML diagrams that a human needs to manually evaluate, the level of involvement in each of the diagrams, in case there is a need to change something.

Improve understanding between stakeholders **vs** the quality of the generated UML diagrams and drawbacks (e.g., confusion) that the low quality of the diagrams can introduce.
Time saved by automatically creating and updating the diagrams **vs** time needed to manually evaluate and fix the diagrams

Informed Decisions



GenAI-related tasks - Finding, configuring and tuning models, and using chatbots and agents.

Stakeholders need to perform new **GenAI-related tasks**.

GenAI assistance - Using GenAI-enabled tools, toolchains and methodologies.

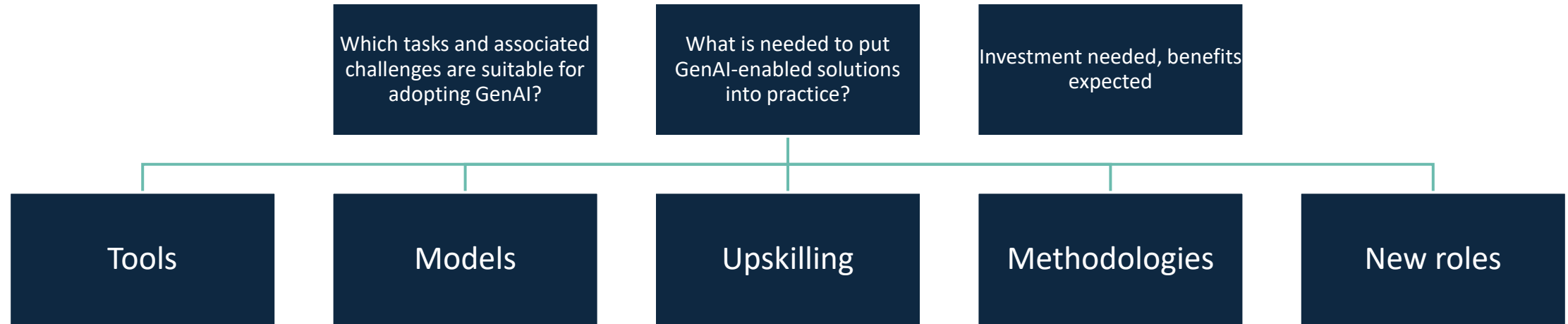
Due to GenAI assistance, the **effort** is **redistributed** (for example, when effort is moved from one stakeholder to another).

GenAI methodologies and toolchains - Integrating tools with GenAI and GenAI methodology with software development methodologies.

There is a **methodology** in place that helps stakeholders, but someone has to create and maintain the methodology.

Do you have relevant skills in your organisation?

Informed Decisions



Assumptions and risks (constraints)

Points in favour of adopting GenAI as an adequate solution in this context, experienced GenAI strengths and beneficial outcomes

Assumptions made for selecting GenAI, risks of the adoption, experienced drawbacks and limitations

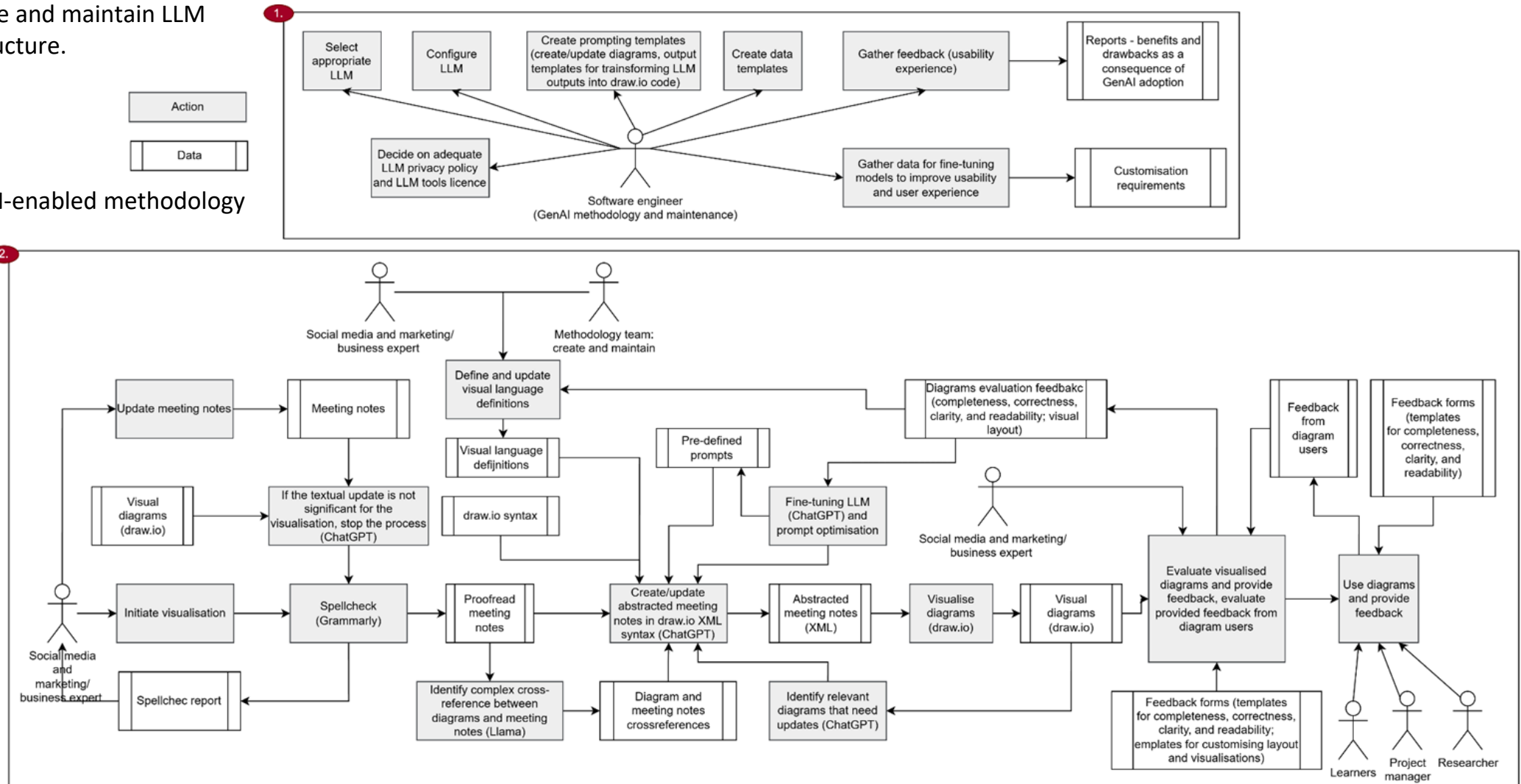
What are the factors that will influence the decision? What can we expect to significantly increase?

As GenAI solves one problem, does it create others?

Informed Decisions – Methodology Example

1. Create and maintain LLM infrastructure.

2. GenAI-enabled methodology



Informed Decisions – Methodology Example

Which tasks and associated challenges are suitable for adopting GenAI?

What is needed to put GenAI-enabled solutions into practice?

Investment needed, benefits expected

Assumptions and risks (constraints)

...

...

All communication between social media and marketing/business expert and other stakeholders will be supported by visualised, summarised, and abstracted previous notes. Social media and marketing/business do not use raw LLMs. Instead, there will be a toolchain in place. One software engineer - methodology team - will create and maintain LLM infrastructure (choose and configure). Methodology team will define a modelling profile in draw.io based on the defined visual language. They will use the defined modelling language to guide creation of prompts (define prompt templates) and use draw.io to visualise generated diagrams (these are generated in XML using draw.io syntax). Syntax and grammar check – use Grammarly. There is a similar tool powered by GenAI: <https://notebooklm.google/>, so this approach is feasible. Training in prompting, knowledge of: structured prompt engineering ([Marvin et al, 2023](#)), prompt engineering best practices ([Arvidsson and Axell, 2023](#); [OpenAI Developers](#)), and, prompt patterns ([White et al, 2023](#)) are also essential. Methodology team will need to acquire skills necessary for the LLM-related tasks (crafting prompts, fine-tuning LLMs), creation of workflows, and creation of templates. Social media and marketing/business expert needs to evaluate visualised notes, hence they need abstraction and visualisation skills. A course on abstractions in software engineering (e.g., software architecture) would help here.

...

...

Business Aspects

Business aspects :

- cost reduction
- productivity gains
- revenue uplift
- time-to-market reduction
- risk avoidance (expected loss reduction)
- develop technology acumen

Costs

- initial investments (one-off)
- operational investments (recurring)

Investment

Initial investments (one-off or upfront)

- Technology: software licences, infrastructure such as cloud setup, integration work and data preparation.
- People: training and upskilling, external consultants and initial hiring.
- Process and governance: security and compliance setup, privacy policy and architecture design.
- Change enablement: pilots and proofs of concept.

Operational investments (ongoing costs)

- Run costs: infrastructure, tokens and licences.
- People: support staff, model or system maintenance and governance roles.
- Risk: legal and regulatory overhead hindering beneficial outcomes.
- Evolution: upgrades, retraining or re-prompting, vendor changes, technical debt, organisation changes and software architecture changes.

Include costs into decision-making process

Decision-making Template

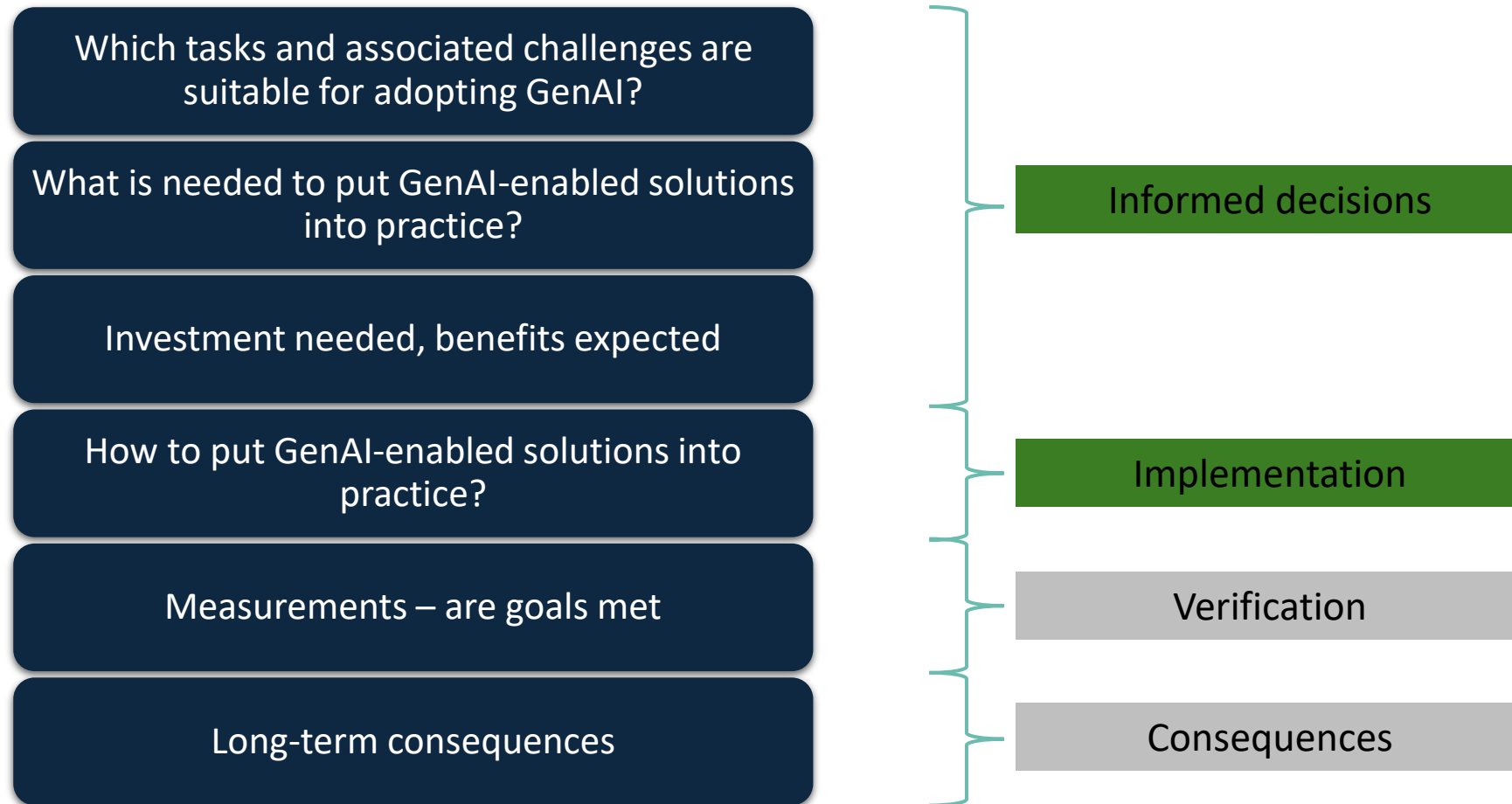
Points in favour of adopting GenAI as an adequate solution in this context, experienced GenAI strengths and beneficial outcomes

Assumptions made for selecting GenAI, risks of the adoption, experienced drawbacks and limitations

What are the factors that will influence the decision?
What can we expect to significantly increase?

As GenAI solves one problem, does it create others?

Top Down Approach



Implementation - How to put GenAI- enabled into Practice?



Unlimited access

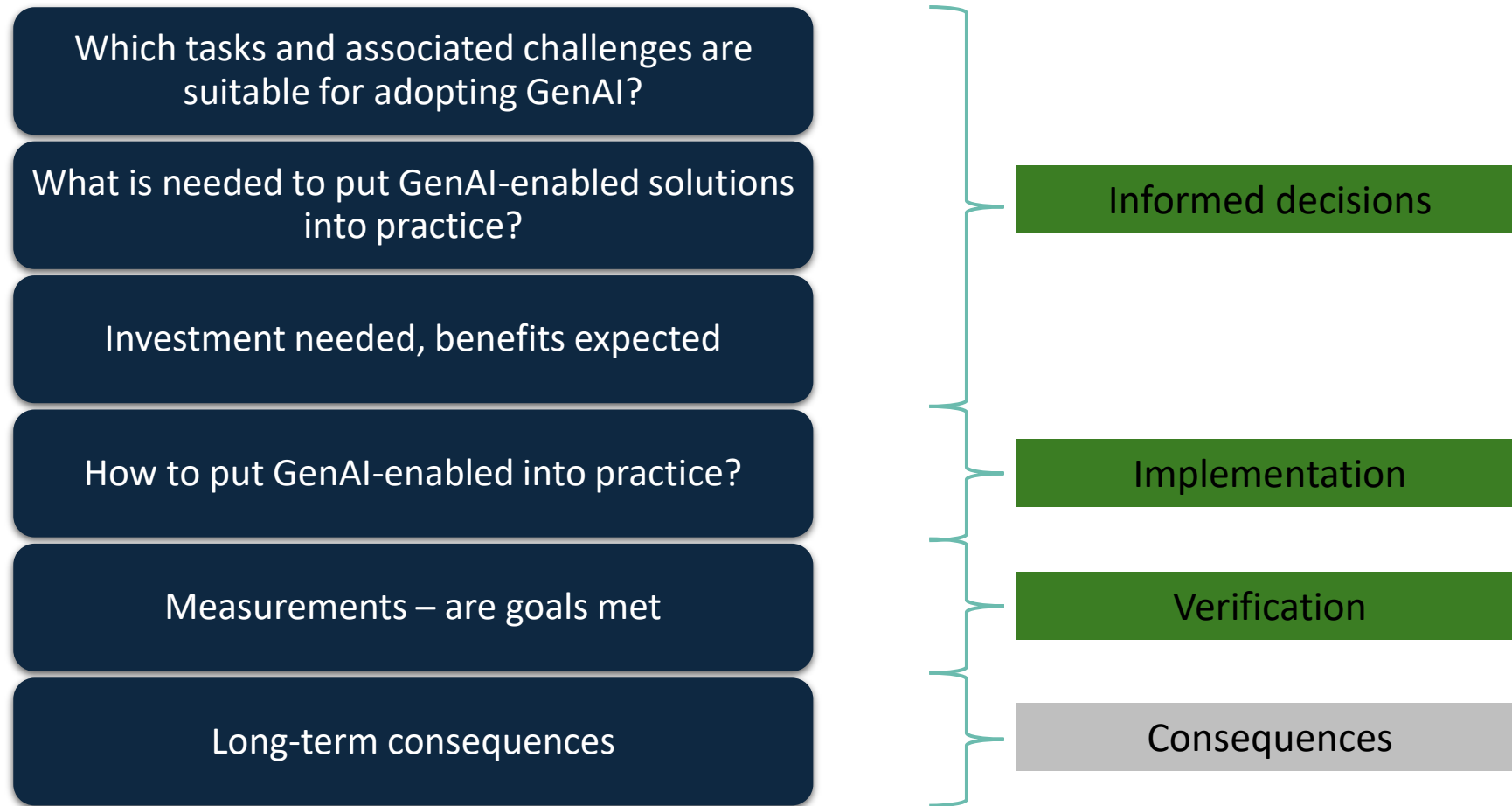


Standardised access
(prompt templates,
data templates)

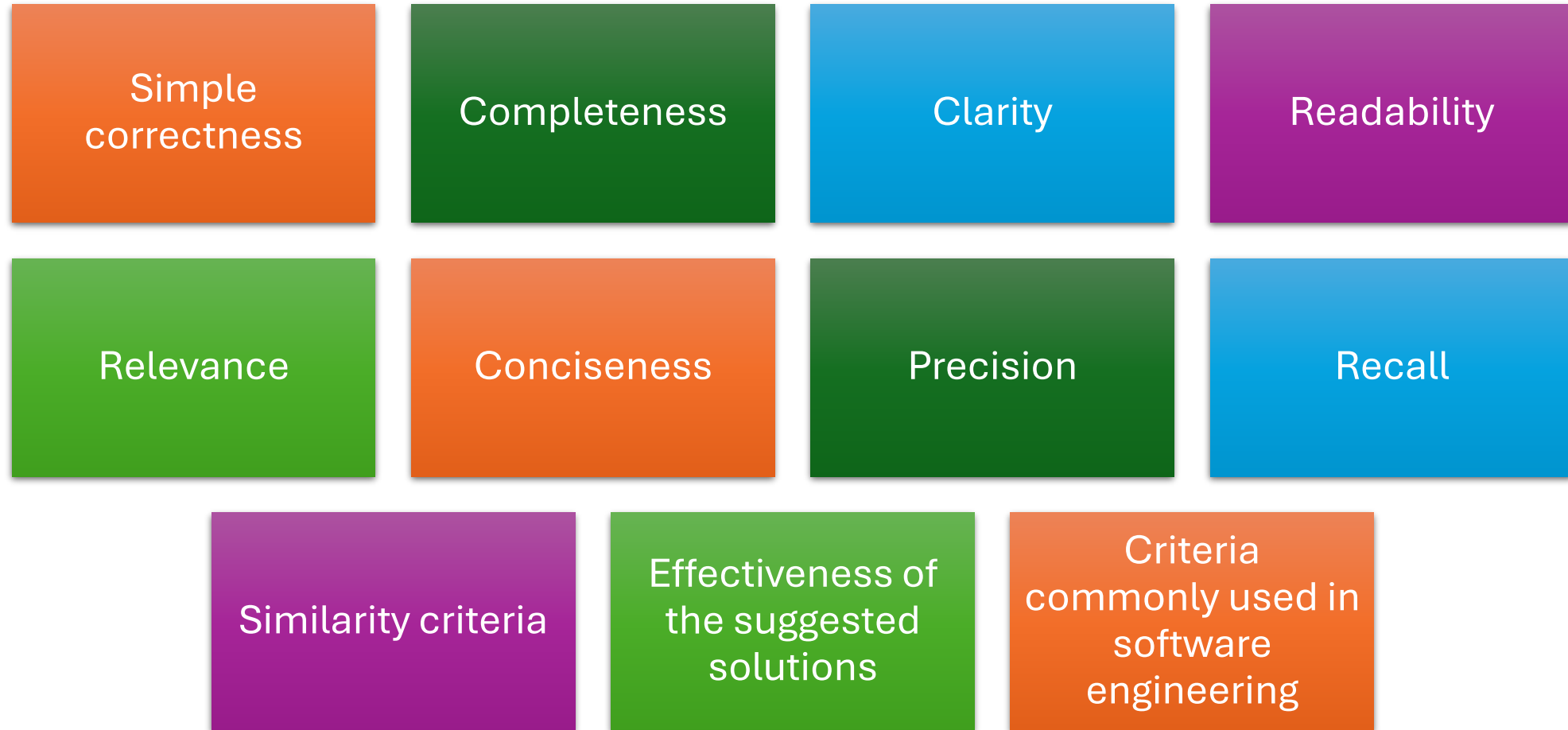


Hidden behind an
interface

Top Down Approach



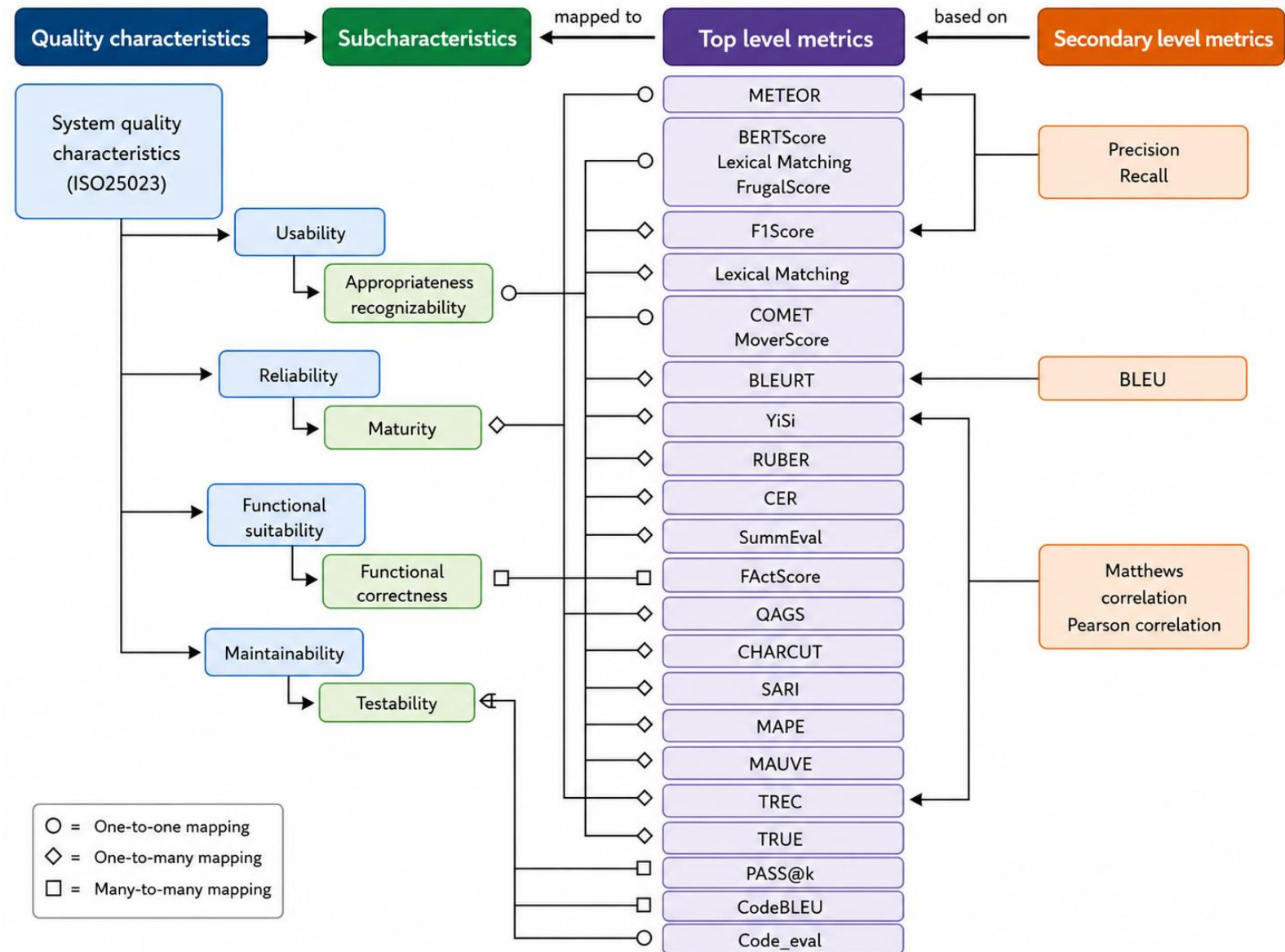
Measuring the Impact of AI Adoption



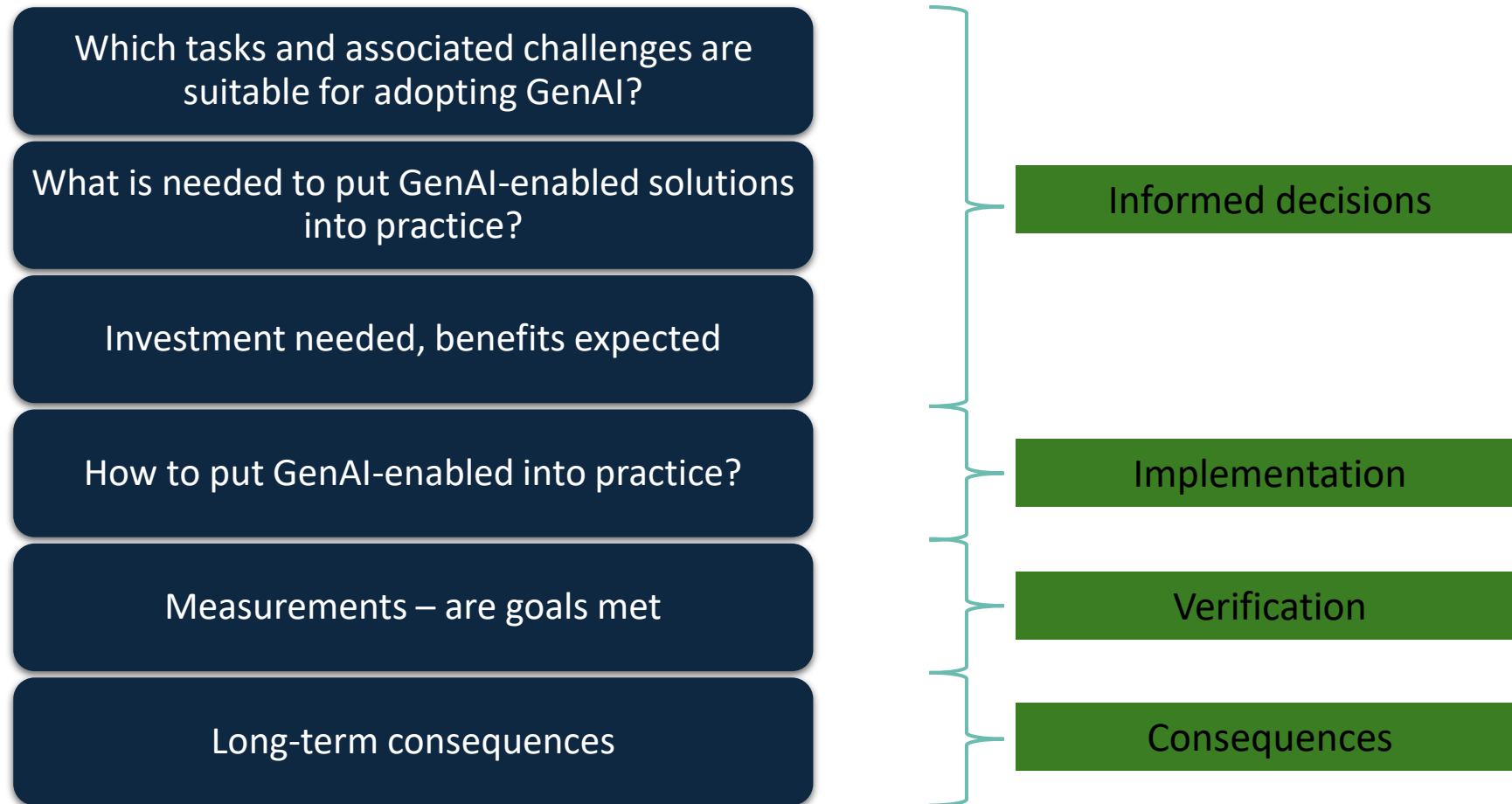
Mapping the identified Metrics to ISO/IEC 25023 Standard for Software Systems

Liang Yu, Emil Alégroth, Panagiota Chatzipetrou, Tony Gorschek, Measuring the quality of generative AI systems: Mapping metrics to quality characteristics — Snowballing literature review, Information and Software Technology, Volume 186, 2025

<https://www.sciencedirect.com/science/article/pii/S0950584925001417>



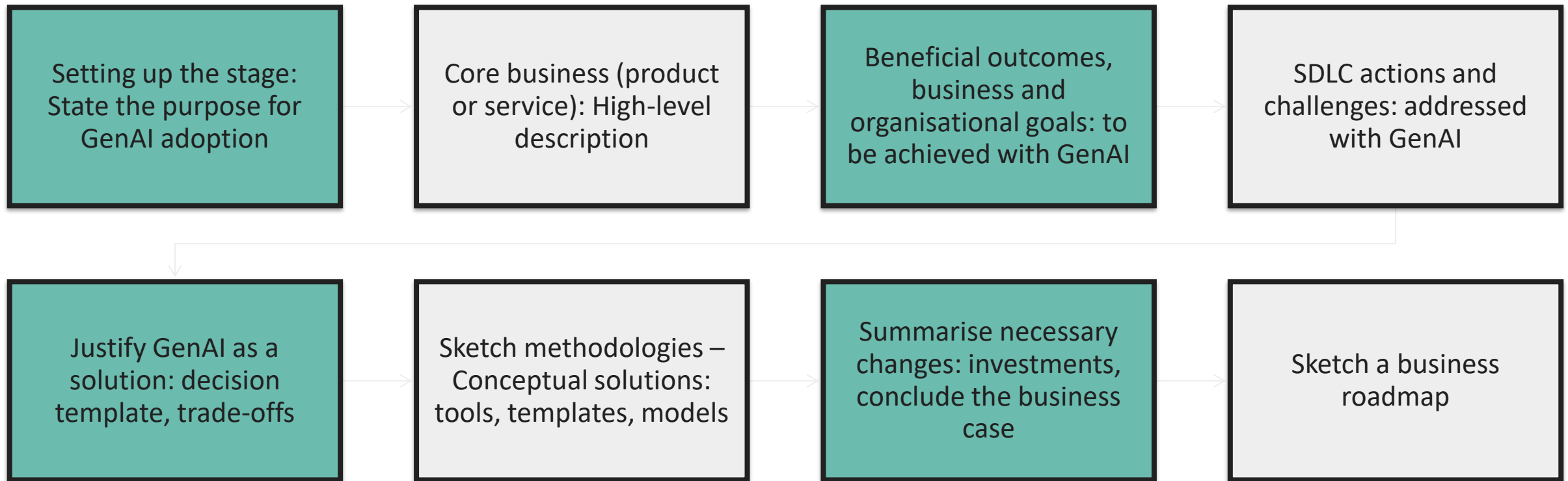
Top Down Approach



Long-term Effects



Summary



Summary

Problem domain: GenAI cannot tell us what we want, can help to shape the problem domain

- E.g., requirements engineering

Solution: GenAI cannot tell us what we want to achieve, can help with suggestions

- E.g., software architecture design

Too many assumptions make solution inadequate